

CyberAudit & Solutions

Final Report:

Project Results Summary, Lessons Learned, Team retrospective highlight

Holberton School Dijon | Tommy JOUHANS & James ROUSSEL | July 2026

Project Name	CyberAudit & Solutions : B2B cybersecurity audit SaaS platform
Project Manager	Tommy JOUHANS (Front-End) / James ROUSSEL (Back-End)
Date	July 6, 2026
Period	Stage 5: July 6th to July 17th, 2026 (5 weeks)
Status	MVP Deployed : Railway (backend) + Vercel (frontend)

1/ Project Completion

This section identifies the criteria required to complete the project and any outstanding items which still need to be undertaken even though the project may be ready for completion.

1.1) Results Summary

CyberAudit & Solutions achieved its MVP objectives within the planned 5 weeks development window. The application is fully deployed and operational in production. Below are the key outcomes compared to the initial objectives.

Initial Objective	Outcome	Status
Full-stack MVP: Django + React	Django REST Framework backend with 5 apps (accounts, audits, reports, training, notifications). React 19 + Vite frontend. Both deployed and accessible.	Achieved
JWT Authentication + RBAC	SimpleJWT with access/refresh tokens, token blacklist on logout, and strict role-based access control (client / admin) on every endpoint.	Achieved
Async PDF generation	Celery + Redis + WeasyPrint pipeline. Admin triggers generation, server responds 202 Accepted, worker generates PDF asynchronously and notifies the client.	Achieved
Test coverage >= 80%	131 backend tests (100% coverage on all 5 apps). 331 frontend tests across 19 Vitest files. CI/CD GitHub Actions enforces both on every push.	Exceeded
Production deployment	Backend live on Railway with PostgreSQL + Redis. Frontend live on Vercel with automatic deploy on push to main. HTTPS enforced on both.	Achieved
Security hardening	DOMPurify + CSP middleware + RBAC + UUID primary keys + anti-enumeration (404 instead of 403) + server-side score recalculation + throttling.	Achieved
E2E Cypress tests	Planned for Sprint 7. Out of scope for the MVP delivery window. Unit and integration tests cover the critical paths.	Deferred

1.2) Key Performance Metrics

Metric	Result
Backend test coverage	100% (131 tests, 1 242 lines covered)
Frontend test coverage	331 tests across 19 Vitest files
Sprint velocity (average)	95% (38 / 40 planned tasks completed)
Open bugs at delivery	0 (all 6 bugs resolved during sprints)
API endpoints	25+ REST endpoints with full RBAC
CI/CD pipeline	Lint + Tests + Build on every PR to main
Time to first deployment	Backend on Railway since May 2026
PDF generation time	2-5 seconds (async, non-blocking)

1.3) Outstanding Items

The items below were identified but remain outside the MVP scope. They are planned for a future version.

Item	Action Required	Owner
E2E Cypress test suite	Implement end-to-end tests covering full user journeys (register, submit audit, download PDF)	Tommy
Data export endpoint (GDPR Art.20)	Add GET /api/auth/me/export returning user data as JSON	James
Auto-deletion of inactive accounts	Celery Beat task: anonymize accounts inactive for 3+ years	James
HSTS preloading	Set SECURE_HSTS_SECONDS=31536000 in Railway environment variables	James
aria-label on icon-only buttons	Add accessible labels to standalone icon buttons for screen readers	Tommy
Login audit log table	Dedicated SQL table for login events (intrusion detection)	James

2/ Lessons Learned

Building CyberAudit taught us a lot in 4 weeks. This section documents what went well, the challenges we encountered and how we resolved them, and the key principles we will carry forward.

2.1) What Went Well

	What Happened	Impact	Future Projects
WIN	Defense in depth security: React auto-escape + DOMPurify + Content Security Policy all active simultaneously.	Zero XSS vulnerabilities reported. CSP blocked external script injection in production (confirmed by test suite).	Apply layered security from day one. Each layer is independent, if one fails, the others still protect.
WIN	Server-side security score recalculation: <code>compute_score_and_grade()</code> in <code>reports/services.py</code> ignores any client-submitted score.	Client cannot manipulate their audit grade. Security score integrity guaranteed regardless of what the frontend sends.	Never trust client input for anything security-critical. Validate and recompute on the server.
WIN	Async by default: Celery + Redis handles all tasks over 1 second. PDF generation, emails, and notifications never block the HTTP thread.	API always responds under 200ms. Users receive 202 Accepted instantly and are notified when the PDF is ready.	Design async from the start for any task that could take more than 1 second. Retrofit is painful.
WIN	Clear front/back split with agreed API contracts before coding. Tommy owned all frontend, James owned all backend.	Zero integration surprises. Mock <code>localStorage</code> let frontend progress without waiting for backend readiness.	Define JSON interfaces and endpoint signatures before coding begins. Eliminates misalignment between teams.

2.2) Challenges and How They Were Resolved

	Challenge	Root Cause	Resolution	Prevention
ISSUE	Railway wiped generated PDFs on every deploy. Reported PDF downloads returned 404 after each new deployment.	Railway's ephemeral filesystem: the container is rebuilt from scratch on each deploy, erasing all files written at runtime.	Attached a persistent Railway volume mounted at MEDIA_ROOT. PDFs now survive deploys.	Any file written at runtime (uploads, generated assets) must use a persistent volume or external storage (S3) in containerized environments.
ISSUE	localStorage caused session conflicts across multiple browser tabs. Logging out in one tab left others still authenticated.	localStorage is shared across all tabs of the same origin. A logout in Tab A did not propagate to Tab B.	Migrated to sessionStorage. Each tab has its own session closing a tab clears it automatically.	Use sessionStorage for auth tokens unless persistent cross-tab state is an explicit requirement. Added security benefit: cleared on tab close.
ISSUE	Our own PDF status polling triggered the server-side rate limiter. Frontend received 429 Too Many Requests while waiting for PDF.	ReportViewerPage polled HEAD /api/reports/pdf/ every few seconds. With multiple open tabs it exceeded the 5 req/min login scope threshold.	Replaced with 30-second smart polling. Stops automatically once pdfReady=true.	Always account for your own polling in rate limit calculations. Exponential backoff or server-sent events are more robust at scale.

2.3) Key Principles for Future Projects

#	Principle	Applied in CyberAudit	How to Apply Next Time
1	Defense in depth	React auto-escape + DOMPurify + CSP: three independent XSS layers. If one fails, the others hold.	Stack multiple independent security controls. Never rely on a single mechanism.
2	Never trust the client	Security score computed server-side. JWT signature verified on every request. RBAC enforced in <code>get_queryset()</code> .	Treat all client input as untrusted. Validate, sanitize, and recompute all security-relevant values on the server.
3	Async by default	Celery handles PDF generation and emails. API always returns in under 200ms.	Any operation over 1 second belongs in a task queue. Design the async path before the synchronous fallback.
4	Agree on contracts before coding	JSON interfaces and endpoint signatures agreed in Stage 3. Frontend used mocks against the same contract.	Write the API contract (request/response shapes, status codes) as a shared document before the first line of code.
5	Test early, test everything	100% backend coverage + 331 frontend tests. CI blocks merges on failing tests.	Set a coverage threshold from sprint 1. Retrofitting tests is 3 times harder than writing them alongside the code.

2.4) Closing Statement

Four weeks. Twenty improvements. One real production MVP. CyberAudit was built under the same constraints as a professional project: two developers, a fixed timeline, a real deployment, and real security requirements. The challenges we faced and resolved, ephemeral filesystems, session management, self-inflicted rate limiting are the same challenges encountered in industry. The principles we derived are not theoretical: they are grounded in production failures and their fixes. We carry them forward.

3/ Team Retrospective

This retrospective was conducted at the end of Stage 4 (July 3, 2026) by the two project members: Tommy JOUHANS (Frontend) and James ROUSSEL (Backend). Its purpose is to evaluate the team's performance honestly, identify what drove success, and define concrete improvements for the next project.

Date	July 3, 2026 (end of Stage 4)
Participants	Tommy JOUHANS / James ROUSSEL
Format	End-of-sprint retrospective: Start / Stop / Continue + 3 guiding questions
Facilitator	Tommy JOUHANS
Note-taker	James ROUSSEL

3.1) What Worked Well as a Team?

Both team members agreed on the following strengths. These are practices to continue and reinforce in future projects.

Practice	Team Observation
Clear ownership split	Tommy owned 100% of the frontend; James owned 100% of the backend. There was never any ambiguity about who was responsible for a given file or decision. This prevented duplicated work and blame-shifting when bugs appeared.
API contract first	Endpoint signatures, request/response shapes, and HTTP status codes were agreed in writing before either side wrote implementation code. The frontend was built against mocks of the same contract, so integration was a formality rather than a negotiation.
CI/CD enforced quality automatically	GitHub Actions ran lint, tests, and build on every pull request. Neither developer could merge a broken branch. This removed the 'it works on my machine' problem entirely and kept the main branch always deployable.
Test coverage from sprint 1	Both team members committed to writing tests alongside code, not after. The result was 100% backend coverage and 331 frontend tests delivered without a crunch at the end. Quality was a habit, not a deadline task.
Async architecture designed early	The decision to use Celery for all operations over 1 second was made in the design phase. This meant the async path was never retrofitted -- it was the default. PDF generation, emails, and notifications all benefited from this upfront decision.
Open communication on blockers	Both developers flagged blockers immediately (Railway crash, rate limit hit, deployment freeze) rather than silently retrying. This allowed the pair to diagnose root causes faster and avoided lost days chasing symptoms.

3.2) What Challenges Did We Face, and How Were They Resolved?

These are the team-level difficulties encountered during the project. Technical root causes are detailed in Section 2.2; this section focuses on the human and process dimension of each challenge.

Challenge	How It Was Resolved	Process Lesson
Infrastructure surprises in production (Railway ephemeral FS, billing crash, GitHub Pages hang)	Each issue was diagnosed from deploy logs, not guesswork. The team adopted a principle: read the logs first, search for the exact error second, never assume. Each fix was documented in the project README immediately.	Production infrastructure should be validated in Sprint 1, not Sprint 5. A 30-minute smoke test on the real platform would have caught the ephemeral filesystem and billing issues three weeks earlier.
Session storage decision (localStorage vs sessionStorage)	The multi-tab bug was reproduced locally, the root cause identified (shared localStorage state), and the fix (sessionStorage) was proposed, reviewed, and merged within a single sprint. The decision was made jointly, not unilaterally.	Security-critical implementation choices (token storage, auth flow) should be reviewed by both team members before the first line of code, not discovered through a bug.
Self-inflicted rate limiting from polling	Once the 429 errors were traced to our own frontend, the team agreed on the fix (30-second polling with auto-stop) in a quick sync call. The fix was merged the same day.	Rate limit thresholds should be written into a shared document at the start of the project. Both frontend and backend developers need to know the limits their code must stay under.
Swagger /api/docs/ returning 404 (drf_spectacular missing from INSTALLED_APPS)	Backend developer identified the mismatch between urls.py and settings.py, added the missing line, and the fix was live within minutes.	Any library imported in code must appear in INSTALLED_APPS or the equivalent config manifest. A checklist of 'code uses it, config declares it' pairs would prevent this class of error.
Merge conflict during integration (Stages/Stage-4/README.md)	Both versions (mermaid diagram from HEAD, PNG image from remote) were examined. The decision to keep both was made immediately, no content was lost. The conflict was resolved in under 10 minutes.	Long-lived feature branches increase merge conflict risk. Shorter-lived branches (max 2 days) and more frequent merges to main would reduce this friction.

3.3) How Can We Improve Collaboration in Future Projects?

The following improvement actions were agreed by both team members. Each has a clear owner and a target milestone.

Improvement Action	Why	Owner	When
Deploy to production in Sprint 1	Infrastructure surprises (ephemeral FS, credentials, CI integrations) are far cheaper to fix at the start than mid-project.	Both	Sprint 1
Cross-review at least one PR per sprint from the other's domain	Each developer currently reviews only their own area. Cross-reviews spread knowledge and catch domain-crossing bugs (e.g., frontend polling against backend rate limits).	Both	Every sprint
Write a shared rate-limits and security-contracts document at kickoff	The polling rate-limit bug happened because the frontend didn't know the backend's throttle thresholds. One shared doc would have prevented it.	James	Sprint 1
Cap feature branches at 2 days before merging to main	Long-lived branches cause merge conflicts and defer integration surprises. Shorter branches keep both developers in sync.	Both	Sprint 2
Add E2E Cypress tests from Sprint 3 (not deferred)	Unit tests confirm individual components work. Only E2E tests confirm the full user journey works end-to-end in production.	Tommy	Sprint 3
Maintain a live .env.example with every required variable documented	The Railway DATABASE_URL rotation caused a production crash partly because the variable mapping was not clearly documented for both team members.	James	Sprint 1
Reserve 10% of each sprint for technical debt and documentation	HSTS, the GDPR export endpoint, and aria-labels were deferred because every sprint was 100% feature work. A dedicated buffer prevents accumulation.	Both	Every sprint

3.4) Documented Feedback

The following individual feedback was recorded verbatim at the close of the retrospective meeting.

From	Feedback
Tommy JOUHANS	Working with a clear contract and a reliable CI pipeline made the frontend work feel solid from day one. I never had to wonder whether my code would break the backend -- the tests and the agreed interfaces handled that. What I would do differently: get Cypress running in week two, not defer it. The lack of E2E tests was the only part of the project that felt unfinished at delivery.
James ROUSSEL	The backend architecture decisions made in Stage 3 -- Celery, RBAC, JWT blacklist -- held up perfectly under pressure. Nothing had to be redesigned mid-sprint. The Railway infrastructure issues were frustrating but educational: containerized environments behave differently from dev machines, and you have to know that before you deploy. For future projects I would insist on a full production smoke test at the end of sprint one, not sprint five.

CyberAudit & Solutions -- Tommy JOUHANS & James ROUSSEL -- Holberton School Dijon 2026